AI-BASED ELECTRICITY DEMAND INTELLIGENCE SYSTEM FOR SRI LANKA

INDIVIDUAL THESIS

Janahan. R

(IT22069740)

Individual component: Adaptive Forecasting and Prediction

Group ID: R26-IT010

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Hons) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Janahan. R

Student ID: IT22069740

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Nathali Silva		

ABSTRACT

Accurate short-term electricity demand forecasts are essential for grid stability, yet most Sri Lankan forecasting studies rely on single recurrent or classical models that return deterministic point estimates and are rarely compared against hybrid or attention-based architectures. This thesis presents the Adaptive Forecasting and Prediction component of the AI-Based Electricity Demand Intelligence System for Sri Lanka (Project R26-IT010). The component forecasts demand 15 minutes ahead from a 24-hour (96-step) window of 20 weather, temporal, calendar and lag features, using a 15-minute dataset spanning January 2020 to May 2025 (189,888 records).

Following a Design Science Research approach, five deep-learning architectures (CNN, LSTM, Transformer, CNN-LSTM and CNN-Transformer) were trained under a chronological 80/10/10 split with training-only scaling, and evaluated as seven variants, including two Monte Carlo (MC) Dropout inference variants. Six variants were compared on an aligned test set of 18,817 rows using MAE, RMSE, MAPE and R^2. The CNN-Transformer with MC Dropout achieved the lowest error (MAE 20.46 kW, RMSE 29.23 kW, MAPE 0.97%, R^2 0.9745), but a paired moving-block comparison found no meaningful difference from the deterministic CNN-LSTM (20.52 kW, $p = 0.90$), which had the lowest peak-period error. MC inference reduced the CNN-Transformer's MAE from 38.80 to 20.46 kW. Calibrating 100-pass MC intervals with a validation-residual quantile gave 96.3% and 97.1% test coverage at the nominal 95% level, with constant widths of 134 and 144 kW.

The variants are served through a FastAPI model registry and dashboard; adaptation means a switchable, ranked registry, not online learning. Limitations include the one-step horizon, a single random seed, undocumented dataset provenance and the absence of stakeholder validation.

Keywords: Electricity demand forecasting, CNN-Transformer, Monte Carlo Dropout, Conformal calibration, Sri Lanka

ACKNOWLEDGEMENTS

I sincerely thank my research supervisor, Dr. Nathali Silva, for their guidance, expert advice and constructive feedback throughout this study. Their mentorship was vital in shaping the project and in helping me address the challenges of AI-based energy forecasting.

I am grateful to the resource personnel and industry experts who shared domain insight, which strengthened the practical relevance of this work, and to the lecturers and staff of the Department of Information Technology for the knowledge, support and resources that made the research possible.

I thank my team members of Project R26-IT010, whose components of the system my forecasting service was integrated with, and my colleagues and friends for their ideas and encouragement.

Finally, I thank my parents and family for their unconditional love, encouragement and sacrifices.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
ABSTRACT ii
ACKNOWLEDGEMENTS iii
TABLE OF CONTENTS iv
LIST OF FIGURES vii
LIST OF TABLES viii
LIST OF ABBREVIATIONS ix
LIST OF APPENDICES xi
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 3
1.3 Problem Statement 4
1.4 Research Gap 5
1.5 Aim and Objectives 5
1.6 Research Questions or Hypotheses 6
1.7 Scope, Limitations and Assumptions 7
1.8 Individual Contribution 8
1.9 Thesis Structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 9
2.2.1 Load forecasting and its horizons 9
2.2.2 Neural architectures for sequence data 10
2.2.3 Uncertainty quantification 10
2.2.4 Concept drift and adaptation 11
2.3 Related Research 11
2.3.1 Forecasting studies in Sri Lanka 11
2.3.2 Convolutional and attention-based models 11
2.4 Existing Systems, Products and Techniques 12
2.5 Gap Synthesis and Conceptual Framework 13
3 RESEARCH METHODOLOGY 15
3.1 Research Design 15
3.2 Data, Participants or Study Materials 15
3.2.1 Input features 17
3.2.2 Sequence construction, splitting and scaling 17
3.3 Experimental or Study Procedure 18
3.3.1 Model variants 19

3.3.2 Uncertainty estimation by MC Dropout 20
3.3.3 Interval calibration from validation residuals 20
3.3.4 Model registry and selection rule 21
3.4 Measures and Evaluation Metrics 21
3.5 Data Analysis 22
3.6 Ethics, Privacy, Security and Responsible AI 22
3.7 Reproducibility and Research Artefacts 23
4 PRODUCT AND SOLUTION DESIGN 25
4.1 Requirements 25
4.1.1 Stakeholders 25
4.1.2 Functional requirements 25
4.1.3 Non-functional requirements 26
4.2 Component Architecture within the Complete Solution 26
4.3 Design Alternatives and Decisions 28
4.4 Detailed Design 29
4.4.1 Input representation 29
4.4.2 Model families 29
4.4.3 Probabilistic inference 30
4.4.4 Interval calibration 30
4.4.5 Registry and selection rule 30
4.4.6 Service interface 30
4.5 Standards and Professional Practice 31
5 IMPLEMENTATION, INTEGRATION AND TESTING 32
5.1 Development Environment and Technology Stack 32
5.2 Component Implementation 32
5.3 Integration with the Other Components 34
5.4 Verification and Software Testing 34
5.5 Deployment and Operation 35
6 RESULTS AND VALIDATION 36
6.1 Research Results 36
6.1.1 Experimental setting 36
6.1.2 Model comparison 36
6.1.3 Relative improvements and significance 37
6.1.4 Peak-period performance 38
6.1.5 Uncertainty and interval quality 39
6.1.6 Predictions on the test set 41
6.2 Product and Technical Results 41
6.3 User or Stakeholder Validation 41
6.4 Integration Results 41

6.5 Summary of Findings 42
7 DISCUSSION 43
7.1 Interpretation of Findings 43
7.2 Comparison with Literature Baselines and Existing Solutions 44
7.3 Research Contribution 45
7.4 Product Contribution 45
7.5 Limitations and Threats to Validity 46
7.6 Implications 47
8 COMMERCIALISATION AND SUSTAINABILITY 48
8.1 Value Proposition and Target Users 48
8.2 Market and Competitive Position 48
8.3 Business or Sustainability Model 48
8.4 Scalability, Intellectual Property and Compliance 49
8.5 Contribution to the Sustainable Development Goals 50
9 CONCLUSION AND FUTURE DIRECTIONS 51
9.1 Conclusion 51
9.2 Achievement of Objectives 52
9.3 Individual Research and Product Contributions 52
9.4 Future Directions 52
REFERENCES 54
GLOSSARY 56
APPENDICES 57
Appendix A Extended Results and Evidence Record 57
A.1 Dataset requirements stated in the project proposal 57
A.2 The 20 input features 57
A.3 Evidence sources in the project repository 59

LIST OF FIGURES

Figure 1.1: High-level architecture of the AI-Based Electricity Demand Intelligence System 3
Figure 3.1: Chronological data split and construction of one supervised sample 18
Figure 3.2: Experimental workflow of the adaptive forecasting component 19
Figure 4.1: Run-time architecture of the adaptive forecasting component 27
Figure 5.1: Dashboard and API output for the forecasting component ..... **Error! Bookmark not defined.**
Figure 6.1: Test MAE by model variant on the aligned test set (lower is better) 37
Figure 6.2: Peak (top-decile) MAE by model variant 39
Figure 6.3: Empirical coverage of raw and calibrated 95% intervals 40

LIST OF TABLES

Table 1.1: Components of the complete solution and the component reported in this thesis 2
Table 1.2: Research questions and associated hypotheses 7
Table 1.3: Individual contributions of the author 8
Table 2.1: Summary of selected related studies and their relationship to this work 12
Table 2.2: Comparison of forecasting technique families 13
Table 2.3: Mapping of research gaps to objectives and design elements 13
Table 3.1: Summary of the dataset and input configuration 16
Table 3.2: The seven model variants and their registry identifiers 20
Table 3.3: Reproducibility checklist and status 24
Table 4.1: Functional requirements of the forecasting component 25
Table 4.2: Non-functional requirements of the forecasting component 26
Table 4.3: Key design decisions and justification 29
Table 5.1: Technology stack and justification 32
Table 5.2: Training configuration of the baseline protocol 33
Table 5.3: Verification checks and available evidence 34
Table 6.1: Test performance of the evaluated variants on the aligned test set (18,817 rows) 36
Table 6.2: MAE reduction achieved by the CNN-Transformer with MC Dropout (20.46 kW) 37
Table 6.3: Peak (top-decile) MAE on the aligned test set 39
Table 6.4: Raw and calibrated 95% interval coverage and mean width 40
Table 6.5: Summary of findings and strength of evidence 42
Table 7.1: Limitations and threats to validity 46
Table 8.1: Planned project budget (LKR) 49
Table 9.1: Achievement of the objectives 52
Table A.1: Dataset requirements stated in the project proposal 57
Table A.2: The 20 input features in the order required by the models 57
Table A.3: Project artefacts that hold the reported evidence 59
Table A.4: Disclosure of generative AI use ..... **Error! Bookmark not defined.**

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ARDL	Autoregressive Distributed Lag
ARIMA	Auto-Regressive Integrated Moving Average
B2G	Business-to-Government
CEB	Ceylon Electricity Board
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DSR	Design Science Research
FR	Functional Requirement
GPU	Graphics Processing Unit
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LECO	Lanka Electricity Company
LKR	Sri Lankan Rupee
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MC	Monte Carlo
ML	Machine Learning
MLP	Multilayer Perceptron
NFR	Non-Functional Requirement
PICP	Prediction Interval Coverage Probability
R^2	Coefficient of determination
REST	Representational State Transfer
RMSE	Root Mean Squared Error

Abbreviation	Meaning
RQ	Research Question
SCADA	Supervisory Control and Data Acquisition
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
SSL	Secure Sockets Layer
XAI	Explainable Artificial Intelligence

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Extended Results and Evidence Record	57

1 INTRODUCTION

1.1 Overall Project Context

Electricity demand is increasing worldwide as industrial activity grows and transport, heating and other end uses are electrified [1]. Power systems must balance supply and demand almost instantaneously, so every improvement in the quality of demand forecasts has a direct operational and financial consequence. In Sri Lanka the Ceylon Electricity Board (CEB) operates a national grid in which demand varies with the time of day, the day of the week, the season, public and religious holidays, and weather. Annual demand has been reported to grow at roughly 5-6% per year over the past decade [2], and the 2022 energy crisis, which involved scheduled power cuts of up to 13 hours a day, showed how costly inadequate demand planning can be [3].

The research project **AI-Based Electricity Demand Intelligence System for Sri Lanka** (Project ID R26-IT010) responds to this need. Rather than offering a single prediction model, the project aims to provide a decision-support system that combines multi-source data integration, behavioural analysis of consumption, advanced deep-learning forecasting and probabilistic uncertainty quantification. The system is implemented in Python, and its services are exposed through a FastAPI backend to a React.js dashboard for energy planners, CEB and Lanka Electricity Company (LECO) operators and analysts.

The work is carried out by a team of four undergraduates, each responsible for one component of the complete solution. Table 1.1 summarises the components and shows where this thesis fits. The overall system architecture adopted by the team is shown in Figure 1.1.

Table 1.1: Components of the complete solution and the component reported in this thesis

Component (service)	Main purpose	Responsibility
Behaviour intelligence service	Clustering of consumption patterns, anomaly detection and behaviour labels	Thanujan T
Adaptive forecasting and prediction (forecast service)	Short-term demand forecasting with hybrid deep learning, uncertainty estimation and calibrated prediction intervals	**Janahan. R (this thesis)**
Explainable AI (XAI) service	Explanations of demand drivers and reports for stakeholders	Madushani R.P
Scenario service	Scenario simulation and decision support for planners	Dewmini D.M.D.P

Source: Project proposal, R26-IT010

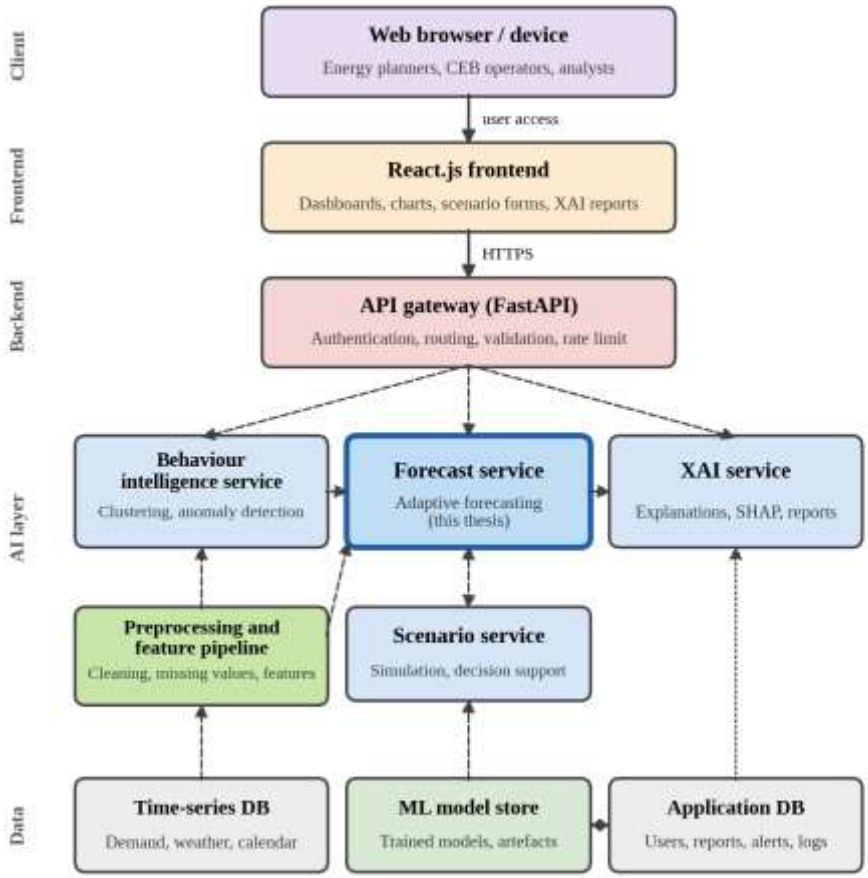


Figure 1.1: High-level architecture of the AI-Based Electricity Demand Intelligence System

Source: Redrawn from the project proposal (R26-IT010). The forecast service, outlined in blue, is the subject of this thesis.

1.2 Individual Component Context

This thesis reports the **Adaptive Forecasting and Prediction** component, which is the forecast service in the AI layer of the architecture. Its task is to predict the next 15-minute electricity demand value from the previous 24 hours of 15-minute observations of weather, temporal, calendar and lag features, to attach a calibrated measure of uncertainty to that prediction, and to make the result available to the rest of the system through an API.

The component receives cleaned, feature-engineered data from the preprocessing and feature pipeline and the time-series database, obtains trained model artefacts from the

model store, and returns forecasts and prediction intervals to the dashboard, to the XAI service and to the scenario service. The behaviour intelligence service supplies information about consumption patterns that can inform when forecasting models should be re-examined.

The word *adaptive* is used in a specific and limited sense in this thesis. The component compares several model families under one evaluation protocol, keeps the resulting variants in a **model registry** with stable identifiers so that the service can switch between them, ranks them by measured error, and attaches calibrated intervals to the Monte Carlo (MC) Dropout variants. The registry has an explicit runtime default (the CNN-LSTM variant) that is a configuration choice, not an automatic accuracy-based decision. The component is therefore a switchable multi-model forecasting framework. It is **not** a system that selects models automatically at run time or continuously updates model weights as new observations arrive, and the thesis does not claim that it is.

1.3 Problem Statement

Short-term demand forecasts support unit commitment, reserve planning and the avoidance of load shedding. Three weaknesses limit the usefulness of the forecasting approaches that have been reported for Sri Lanka [4], [5], [6]:

- **Limited model families.** Published Sri Lankan studies concentrate on LSTM and related recurrent or classical models. Systematic comparisons with convolutional, attention-based and hybrid architectures are scarce.
- **Deterministic outputs.** Forecasts are normally reported as point values judged by MAE and RMSE, with no statement of how uncertain each value is. Planners who must hold reserve margins need a range, not only a single number.
- **Static models.** A model trained once on historical data gradually loses relevance when consumption behaviour changes, yet existing studies rarely describe how the model in use is chosen, checked or replaced.

The problem addressed by this thesis is therefore: *how can 15-minute electricity demand in Sri Lanka be forecast accurately by systematically comparing hybrid*

deep-learning architectures, while attaching calibrated uncertainty to each forecast so that the output can support operational decisions?

1.4 Research Gap

The project proposal identified five technical gaps in the Sri Lankan electricity-forecasting literature. Those that concern the forecasting component are restated here, together with the way in which this thesis responds to each.

- **Architectural stagnation.** Studies specific to Sri Lanka [4], [5] rely mainly on LSTM and multilayer perceptron (MLP) models or on gradient boosting. To the best of the author's knowledge at the time of the proposal's literature survey, no published Sri Lankan study had evaluated CNN-LSTM or Transformer-based hybrids on local demand data.
- **Absence of hybrid and ensemble comparisons.** Architectures that combine local feature extraction with sequence modelling or attention are rarely compared against their single-family counterparts under identical conditions.
- **Missing probabilistic forecasting.** Accuracy metrics such as MAE and RMSE say nothing about forecast uncertainty, which is increasingly important for risk-based planning [7].
- **Static forecasting.** There is little published guidance on how a forecasting component should compare candidate models and select between them as conditions change [8].

The remaining gaps identified in the proposal, namely integration of heterogeneous external data sources, behavioural analysis and interpretability, belong primarily to the other three components of the project and are outside the scope of this thesis, although the forecasting component is designed to interoperate with them.

1.5 Aim and Objectives

Aim. To design, implement and evaluate an adaptive short-term electricity-demand forecasting framework for Sri Lanka that compares heterogeneous deep-learning architectures under a common chronological protocol, quantifies predictive

uncertainty and delivers calibrated forecasts as a service to the wider intelligence system.

The aim is pursued through the following objectives:

1. Construct a leakage-safe forecasting data pipeline that converts 15-minute demand records (2020-2025) into supervised sequences (96-step lookback, one-step horizon, 20 features) using a chronological 80/10/10 split and training-only scaling.
2. Train five deep-learning architectures (CNN, LSTM, Transformer, CNN-LSTM and CNN-Transformer) and evaluate seven variants, including two MC Dropout inference variants, using MAE, RMSE, MAPE, R^2 and a peak-period error measure.
3. Estimate predictive uncertainty with MC Dropout (100 stochastic forward passes) and calibrate 95% prediction intervals using validation residuals.
4. Provide a model registry with stable variant identifiers, a documented ranking procedure and an explicit runtime default, which together form the adaptation logic of the component.
5. Expose forecasts and intervals through a FastAPI backend and integrate them with the project dashboard.
6. Critically evaluate the results, documenting limitations, failure modes and threats to validity.

1.6 Research Questions or Hypotheses

The study is guided by the following research questions (RQ) and hypotheses (H).

Table 1.2: Research questions and associated hypotheses

ID	Research question	Hypothesis
RQ1	Which of the evaluated architectures gives the lowest one-step-ahead error on the same chronological test set?	H1: A hybrid architecture (CNN-LSTM or CNN-Transformer) achieves lower test MAE than any single-family model (CNN, LSTM or Transformer).
RQ2	Does adding self-attention to a convolutional front end improve on a convolutional-recurrent hybrid?	H2: The CNN-Transformer achieves lower test MAE than the CNN-LSTM, and the difference is examined with a paired statistical comparison.
RQ3	Can MC Dropout combined with residual conformal calibration produce usable prediction intervals?	H3: Calibrated intervals reach close to their nominal 95% coverage on held-out data.
RQ4	Can the forecasting capability be delivered as a service that other components can consume?	H4: Forecasts and intervals can be served through a FastAPI backend and displayed on the dashboard.

1.7 Scope, Limitations and Assumptions

In scope: one-step (15-minute) ahead forecasting of Sri Lankan electricity demand from a 24-hour window of 20 features; comparison of seven model variants; MC Dropout uncertainty estimation; validation-residual calibration of 95% intervals; a model registry with ranking-based selection; and integration through FastAPI and the dashboard.

Out of scope: continuous online learning with weight updates after every observation; automatic, rule-driven model switching at run time; multi-horizon and long-term forecasting; causal analysis of weather and socio-economic drivers (the responsibility of the XAI and data-integration work); consumption clustering and anomaly detection (the behaviour intelligence component); and deployment on a live utility production system.

Assumptions: the historical series is representative of the demand the component will meet in operation; the supplied data are aggregated, anonymised and representative of real demand, although their original provenance is not documented; the chronological ordering of the data is reliable; and dropout-based stochasticity gives a useful, though approximate, indication of predictive uncertainty. The main limitations are discussed in Section 7.5.

1.8 Individual Contribution

The individual contributions of the author to the project are summarised in Table 1.3.

Table 1.3: Individual contributions of the author

Area	Contribution
Data preparation	Design of the 96-step, 20-feature, one-step supervised dataset; chronological 80/10/10 split; training-only scaling to avoid information leakage
Model development	Training of five architectures and comparison of seven variants, including two MC Dropout inference variants
Uncertainty quantification	MC Dropout inference (100 passes) and validation-residual calibration of 95% prediction intervals
Adaptive logic	Model registry with stable variant IDs, MAE-based ranking and an explicit runtime default
Integration	FastAPI backend exposure of forecasts and dashboard integration with the other project components
Evaluation	Aligned-test evaluation with MAE, RMSE, MAPE, R^2, peak-period error, a paired significance comparison, interval-coverage analysis and documentation of limitations

1.9 Thesis Structure

Chapter 2 reviews the literature on electricity-demand forecasting, deep-learning architectures, uncertainty quantification and adaptation to changing conditions, and synthesises the research gap. Chapter 3 describes the research methodology, data, procedure, evaluation metrics and ethical considerations. Chapter 4 presents the requirements and design of the forecasting component within the complete solution. Chapter 5 describes implementation, integration and testing. Chapter 6 reports results and validation, and Chapter 7 discusses them, including contributions, limitations and implications. Chapter 8 addresses commercialisation and sustainability. Chapter 9 concludes the thesis and proposes future directions. References, a glossary and appendices follow.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review is a targeted narrative review intended to position the component, rather than an exhaustive systematic survey. The search was organised around the keywords *electricity demand forecasting*, *AI-based smart grid management*, *electric load pattern analysis*, *Ceylon Electricity Board electricity demand* and *energy demand prediction using LSTM*, and extended to uncertainty quantification, conformal prediction and concept drift. Priority was given to studies of Sri Lankan demand, to deep-learning and hybrid models, and to probabilistic forecasting.

Specific statements about individual studies in this chapter are limited to what those studies report in their titles and abstracts as recorded in the project proposal. Numerical results from different studies are not directly comparable because they use different datasets, horizons and spatial levels, and they are not compared numerically in this thesis.

2.2 Domain and Theoretical Foundations

2.2.1 Load forecasting and its horizons

Electricity cannot be stored cheaply at grid scale, so generation must follow demand. Load forecasts are customarily grouped by horizon: very short-term (minutes to hours), short-term (hours to days), medium-term (weeks to months) and long-term (years). The component in this thesis operates at the very short-term end, predicting one 15-minute interval ahead, which is relevant to dispatch, reserve and balancing decisions.

Demand is hard to forecast because it combines several overlapping seasonalities (daily, weekly and annual), weather dependence, calendar effects such as public holidays and Poya days, structural change (new appliances, rooftop solar, electric vehicles, tariff changes), irregular events, and data-quality problems such as missing readings and meter faults.

2.2.2 Neural architectures for sequence data

Convolutional neural networks (CNN). One-dimensional convolutions slide learnable filters across a time window and extract local patterns such as ramps and short-term fluctuations. They are fast to train and effective at local feature extraction, but a stack of convolutions has a limited receptive field.

Long short-term memory (LSTM). LSTM networks use gated memory cells to retain information across many time steps and mitigate the vanishing-gradient problem of simple recurrent networks [9]. They are the most widely used deep model in the load-forecasting literature, but they process the sequence step by step and can be slow to train on long windows.

Transformer. Transformers replace recurrence with self-attention, allowing every position in the input window to attend directly to every other position [10]. Attention is well suited to long-range dependencies, but without a local inductive bias it can need more data and careful design to match recurrent or convolutional models on short, noisy windows.

Hybrid models. Hybrids combine the strengths of these families. In a CNN-LSTM, convolutional layers extract local patterns that an LSTM then models over time. In a CNN-Transformer, convolutional layers provide the local representation and self-attention models relationships across the window. Hybrids are the central subject of the comparison in this thesis.

2.2.3 Uncertainty quantification

A point forecast hides how uncertain the prediction is. Probabilistic load forecasting provides quantiles, intervals or full distributions and is increasingly regarded as essential for risk-aware operation [7]. Two techniques are used in this work.

Monte Carlo (MC) Dropout. Gal and Ghahramani showed that a network trained with dropout can be interpreted as an approximate Bayesian model, and that keeping dropout active at prediction time and repeating the forward pass yields a distribution of predictions whose spread reflects model uncertainty [11]. The approximation is practical but not exact, and its spread is not guaranteed to match the true error distribution.

Conformal calibration. Conformal prediction converts the residuals of a fitted model on held-out calibration data into prediction intervals with finite-sample coverage guarantees under an exchangeability assumption [12], [13]. It is attractive here because it can correct the miscalibration of a neural network's raw uncertainty estimates. Time-series data are not strictly exchangeable, so the guarantee is approximate in this setting and coverage must be checked empirically.

2.2.4 Concept drift and adaptation

When the relationship between inputs and demand changes, the accuracy of a model trained on older data degrades. This phenomenon is called concept drift and may be sudden, gradual, incremental or recurring [8]. Adaptation strategies include periodic or sliding-window retraining, online learning, drift-triggered retraining and dynamic ensembles or model switching. This thesis implements a limited form of the last of these, a registry of alternative trained models from which the service can be switched, and treats the others as future work.

2.3 Related Research

2.3.1 Forecasting studies in Sri Lanka

Several studies have applied machine learning to Sri Lankan demand. Jayasinghe et al. applied deep-learning techniques to electricity load forecasting and reported improved accuracy compared with conventional approaches [5]. Shiwakoti and co-authors compared statistical and deep-learning models for short-term forecasting and found that the LSTM produced the lowest errors for next-day demand [6]. A further study combined the Prophet forecasting model with XGBoost using four years of CEB data and reported that the hybrid reduced the MAE by 19.45 and the MAPE by 1.32% relative to Prophet alone [4]. Priyadarshana et al. used an autoregressive distributed lag (ARDL) model with weather variables and found it the most accurate of eight methods for monthly consumption, underlining the influence of environmental factors [14].

2.3.2 Convolutional and attention-based models

Karunarathne et al. used convolutional neural networks for microgrid demand forecasting and reported lower MAE and RMSE than the models they compared

against [15]. Nazir et al. forecast customer energy consumption in a smart grid with the Temporal Fusion Transformer [16], illustrating the interest in attention-based architectures. Other recent studies apply LSTM and general machine-learning models to demand forecasting [17], [18].

Table 2.1: Summary of selected related studies and their relationship to this work

Study	Approach	Context	Relationship to this thesis
[5]	Deep-learning load forecasting	Sri Lanka	Local evidence for deep learning; single-model point forecasts
[6]	Statistical vs deep-learning comparison	Sri Lanka, next-day	LSTM strongest in that comparison; no hybrid or probabilistic output
[4]	Prophet-XGBoost hybrid	CEB data, four years	Shows value of hybrids; classical rather than deep hybrid
[14]	ARDL with weather variables	Sri Lanka, monthly	Highlights weather effects; monthly rather than 15-minute
[15]	CNN forecasting	Microgrid	Supports convolutional feature extraction
[16]	Temporal Fusion Transformer	Smart-grid customers	Attention-based forecasting; not Sri Lankan data
[17], [18]	LSTM and ML forecasting	General	Background evidence for neural forecasting

2.4 Existing Systems, Products and Techniques

Existing techniques fall into three broad families. Statistical models such as ARIMA and ARDL are transparent and cheap, but they struggle with the non-linear interactions among time, weather and calendar effects. Classical machine-learning models, including gradient boosting and hybrids such as Prophet-XGBoost, handle non-linearity and external variables well but rely on manual feature design and typically produce point forecasts. Deep-learning models learn features directly from sequences and have given the strongest accuracy in recent studies, but they are more demanding in data and computation and are less transparent. Table 2.2 compares the families against the requirements of this project.

Table 2.2: Comparison of forecasting technique families

Family	Strengths	Limitations for this project
Statistical (ARIMA, ARDL, regression)	Interpretable, low cost, established	Limited non-linearity; weak at 15-minute resolution with multiple seasonalities
Classical ML (XGBoost, Prophet-hybrids)	Strong tabular performance; handles external variables	Manual feature engineering; usually point forecasts
Single deep models (CNN, LSTM, Transformer)	Learn temporal structure automatically	Each captures only part of the structure; deterministic by default
Hybrid deep models (CNN-LSTM, CNN-Transformer)	Combine local and long-range structure	Higher complexity; limited local evidence
Probabilistic add-ons (MC Dropout, conformal)	Provide intervals and risk information	Approximate; require calibration and empirical coverage checks

2.5 Gap Synthesis and Conceptual Framework

The reviewed literature shows that deep learning and hybrid methods generally outperform traditional statistical approaches, that external factors matter, and that probabilistic outputs are valuable. It also shows that, for Sri Lanka, hybrid CNN-LSTM and Transformer-based architectures, probabilistic forecasting and explicit model-selection logic have received little attention. Table 2.3 links each gap to the objective and design element of this thesis that responds to it.

Table 2.3: Mapping of research gaps to objectives and design elements

Gap	Objective	Design element
Limited model families and no hybrid comparison	Objective 2	Seven-variant comparison under one protocol
Deterministic point forecasts	Objective 3	MC Dropout with 100 passes and validation-residual calibration
Static, non-adaptive forecasting	Objective 4	Model registry with ranking-based selection and explicit default
Research prototypes not delivered as services	Objective 5	FastAPI backend and dashboard integration
Risk of optimistic evaluation	Objective 1	Chronological split and training-only scaling

The resulting conceptual framework treats forecasting as a pipeline from a 24-hour feature window, through a pool of candidate neural models, to a calibrated probabilistic forecast served to users. It is realised in the design of Chapter 4.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study follows **Design Science Research (DSR)** [19], in which an artefact is built and evaluated to solve an identified problem. The artefact here is the adaptive forecasting component. The DSR cycle was applied as problem identification (Chapter 1), definition of requirements and objectives (Chapters 1 and 4), design and development (Chapters 4 and 5), demonstration through the API and dashboard (Chapter 5) and evaluation (Chapter 6).

Evaluation is quantitative and experimental. Several model architectures are trained on the same data with the same preprocessing and compared on the same held-out test rows, so that differences in error can be attributed to the model rather than to the data or protocol. The approach is comparative and applied: it asks which design is most suitable for the Sri Lankan forecasting task rather than testing a universal theory.

3.2 Data, Participants or Study Materials

No human participants were involved. The study material is the *Sri Lanka electricity demand dataset*, a CSV file (`electricity_demand_srilanka_fixed.csv`) obtained from Kaggle [20]. The project repository records the file as the single source used by all model notebooks, but it does not record an original publisher, collection method or download address beyond the supplied file. Table 3.1 summarises the data configuration.

Table 3.1: Summary of the dataset and input configuration

Item	Configuration
Dataset	Sri Lanka electricity demand dataset (`electricity_demand_srilanka_fixed.csv`)
Source	Kaggle [20]; original publisher and collection method not documented
Date range	1 January 2020 00:00 to 31 May 2025 23:45 (1,978 days, about 5.4 years)
Number of records	189,888 rows
Sampling frequency	15 minutes
Target variable	Load Demand (kW)
Lookback window	96 steps = 24 hours
Forecast horizon	1 step = next 15 minutes
Input dimensionality	20 features: 5 weather, 8 temporal, 2 calendar/event, 5 lag/rolling
Split	Chronological: 80% training, 10% validation, 10% test
Scaling	Fitted on the training set only and applied unchanged to validation and test
Aligned test set	18,817 rows common to the six variants with saved predictions

The date range and row count are consistent: 1,978 days at 96 observations per day give exactly 189,888 records, so the series appears to be complete and regular. Two observations about the data should be recorded. First, the reported MAE (20.46 kW) and MAPE (0.97%) together imply a typical demand level of the order of 2,000 kW, which suggests that the series does not describe national-scale demand; its spatial level is not stated in the available documentation. Second, the project proposal planned to use data from government and energy-authority sources and to include socio-economic variables such as GDP and electricity price. The dataset actually used does not contain socio-economic variables, and its relationship to Ceylon Electricity Board (CEB) measurements cannot be verified. These points are treated as limitations in Section 7.5.

3.2.1 Input features

The 20 features, listed in the order required by the models, fall into four groups (Table A.2 in Appendix A gives the full list). **Weather** (5): temperature, humidity, wind speed, rainfall and solar irradiance. **Temporal** (8): sine and cosine encodings of hour of day, day of week and month, a weekend indicator and a normalised year index. **Calendar and event** (2): a public-event flag and a Poya-day flag. **Lag and rolling** (5): demand lagged by 96, 192 and 672 steps (1, 2 and 7 days), a weekly rolling mean and a daily rolling standard deviation.

The feature set contains no raw current-demand channel, so recent demand enters the model only through the lag and rolling features. It is assumed that these features are aligned so that no value from the target step or later is included; this should be confirmed against the feature-engineering code. The specific cleaning rules used to produce the supplied "fixed" file are not documented in the repository; the thesis therefore treats the file as given.

3.2.2 Sequence construction, splitting and scaling

Supervised samples are created with a sliding window. Each sample consists of 96 consecutive observations of the 20 features (a tensor of shape 96 × 20) and a target equal to the demand in the next 15-minute interval. The series is divided **chronologically** into 80% training, 10% validation and 10% test segments, as illustrated in Figure 3.1. Random shuffling and ordinary k-fold cross-validation are avoided because they would allow information from the future to influence training, and training batches are not shuffled. Feature-scaling parameters are fitted on the training segment only and then applied unchanged to the validation and test segments, which prevents leakage of test-period statistics.

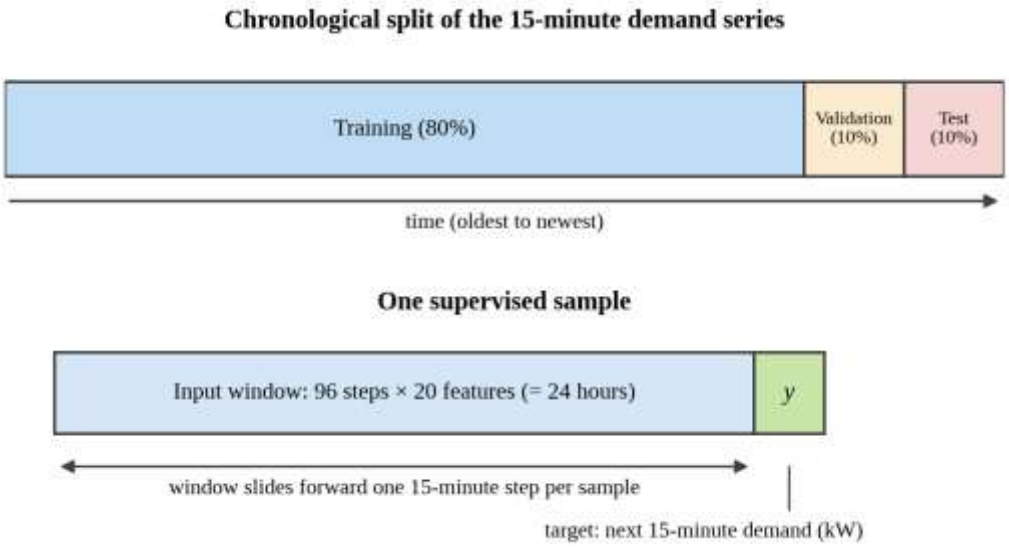


Figure 3.1: Chronological data split and construction of one supervised sample

Source: Author's design based on the experimental protocol.

3.3 Experimental or Study Procedure

The experimental workflow is shown in Figure 3.2. Training and evaluation are performed offline in notebooks. The resulting artefacts are registered in a model registry and served through the FastAPI backend.

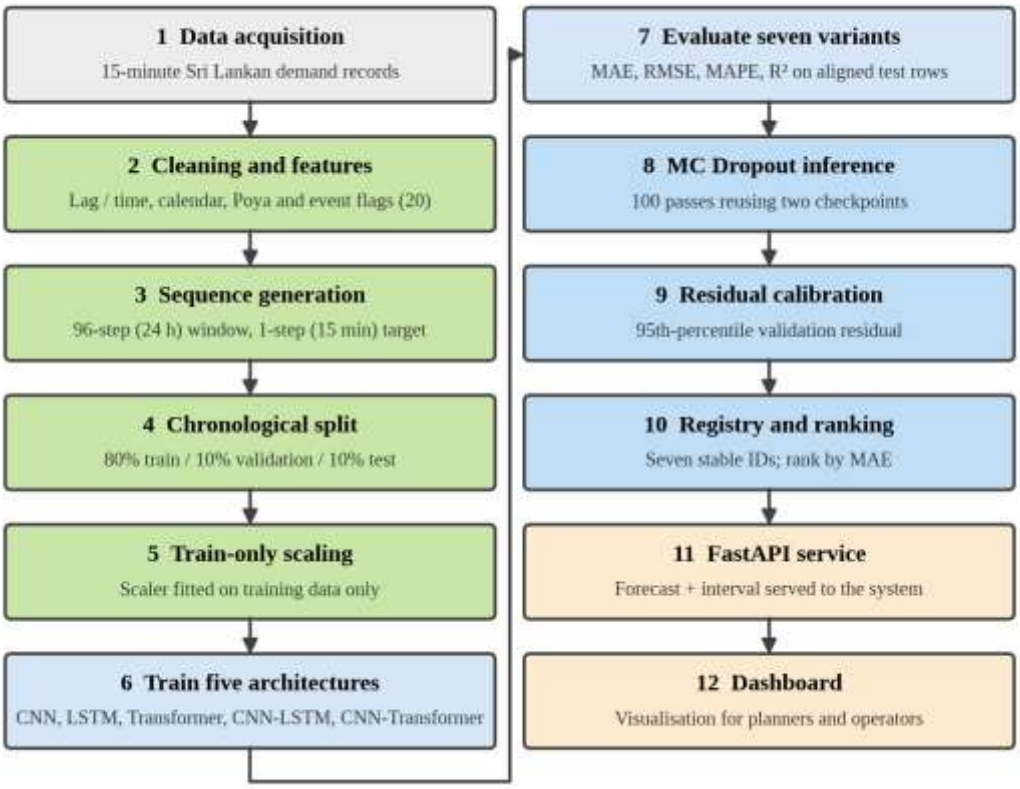


Figure 3.2: Experimental workflow of the adaptive forecasting component

Source: Author's design.

3.3.1 Model variants

Five architectures were trained: a CNN, an LSTM, a Transformer, a CNN-LSTM and a CNN-Transformer. Two further variants, the CNN-LSTM with MC Dropout and the CNN-Transformer with MC Dropout, reuse the checkpoints of the CNN-LSTM and CNN-Transformer and differ only at inference, where dropout is kept active over repeated stochastic passes. The result is seven variants, each with a stable identifier in the registry (Table 3.2). A capacity-matching CNN-Transformer experiment (V2) was tried and rejected for substantial underfitting; it is not one of the seven finalised variants.

Table 3.2: The seven model variants and their registry identifiers

Registry ID	Variant	How it is obtained
cnn	CNN	Trained checkpoint
lstm	LSTM	Trained checkpoint
transformer	Transformer	Trained checkpoint
cnn_lstm_b	CNN-LSTM (baseline, deterministic)	Trained checkpoint
cnn_transformer_b	CNN-Transformer (baseline, deterministic)	Trained checkpoint
cnn_lstm_mc	CNN-LSTM with MC Dropout	Reuses the cnn_lstm_b checkpoint; stochastic inference
cnn_transformer_mc	CNN-Transformer with MC Dropout	Reuses the cnn_transformer_b checkpoint; stochastic inference

Training settings are given in Table 5.2 (Section 5.2). Layer-level dropout rates differ between architectures and are recorded in each notebook and checkpoint.

3.3.2 Uncertainty estimation by MC Dropout

For the MC Dropout variants, dropout layers remain active at inference. Each input is passed through the network 100 times in the research notebooks, giving a set of stochastic predictions whose mean is used as the point forecast and whose standard deviation expresses model uncertainty. The production runtime can be configured with a different number of passes, which should not be confused with the number of training epochs.

3.3.3 Interval calibration from validation residuals

Raw dropout spread is not guaranteed to have the desired empirical coverage, so it is calibrated using residuals. For each MC checkpoint the procedure is:

1. Run the checkpoint on the validation sequences with dropout active for 100 stochastic passes and take the MC mean as the point forecast.
2. Compute the absolute validation residuals in kW, $r_i = |y_i - \text{mean}_{MC}(x_i)|$.

3. Set $\alpha = 0.05$ and take the empirical 95th percentile of the residuals as a single scalar $\hat{q}$ (66.997 kW for the CNN-LSTM and 72.048 kW for the CNN-Transformer).
4. Form the nominal 95% interval $[\text{mean}_{MC}(x) - \hat{q}, \text{mean}_{MC}(x) + \hat{q}]$. The deployment configuration stores $\hat{q}$ (q_hat_kw) and the runtime adds and subtracts it from the prediction.

This follows the logic of split-conformal prediction [12], [13] but is not strictly equivalent to it. The validation set was also used for early stopping, and the saved evaluation files describe the calibrated coverage as supplementary rather than as a strict conformal guarantee. The exact quantile index and interpolation convention are not recorded in the saved artefacts. The interval has constant width because it is a single scalar added to every forecast.

3.3.4 Model registry and selection rule

The adaptation logic is explicit and deliberately simple:

- The seven variants are the explicit entries of a model registry (`api/core/model_registry.py`). A request names a variant by its stable ID, and an unavailable model is never silently replaced by another.
- For comparison, metrics are computed on the common timestamp-aligned test rows and comparable variants are ranked by ascending MAE. The CNN-LSTM with MC Dropout is excluded from this ranking because its prediction file is unavailable.
- The backward-compatible runtime default is `cnn_lstm_b`. This is a configuration choice, not an accuracy-based fallback, and no rule that selected it from validation scores is documented. No exhaustive hyperparameter or model search is documented either, and none should be inferred.

3.4 Measures and Evaluation Metrics

Point forecasts are evaluated with four metrics. Let y_t be the observed demand, $\hat{y}_t$ the forecast and n the number of test samples.

$$\text{MAE} = (1/n)\ \Sigma\ |y_t - \hat{y}_t| \tag{3.1}$$

$$\text{RMSE} = \sqrt{[\ (1/n)\ \Sigma\ (y_t - \hat{y}_t)^2\ ]} \quad (3.2)$$

$$\text{MAPE} = (100\%/n)\ \Sigma\ |y_t - \hat{y}_t| / |y_t| \quad (3.3)$$

$$R^2 = 1 - \Sigma\ (y_t - \hat{y}_t)^2 / \Sigma\ (y_t - \bar{y})^2 \quad (3.4)$$

MAE ((3.1)) is the average absolute error in kW and is the primary measure. RMSE ((3.2)) penalises large errors more strongly. MAPE ((3.3)) gives a relative error but becomes unstable when actual demand approaches zero. R^2 ((3.4)) is the proportion of variance explained relative to a mean predictor.

Peak error is the MAE computed only on observations whose actual load is at or above the 90th percentile of the aligned test actual load (the top decile). It shows how well a model follows high-demand periods, which aggregate metrics can hide. No separate holiday-only metric was computed.

Uncertainty is evaluated with the **empirical coverage** of the 95% interval (the fraction of observations that fall inside it, also called the prediction interval coverage probability, PICP) and the **mean interval width**. A narrow interval is only useful if it also achieves its nominal coverage.

3.5 Data Analysis

All variants with saved predictions are compared on the same aligned test set of 18,817 rows. The analysis comprises (a) comparison of MAE, RMSE, MAPE and R^2; (b) comparison of peak (top-decile) MAE; (c) a paired moving-block comparison of the two leading models, which accounts for the serial correlation of time-series errors; and (d) evaluation of raw and calibrated interval coverage and width for the MC Dropout variants. All runs use one random seed (42), so run-to-run variability from re-training was not measured. Learning-curve and residual-distribution plots were not retained in the project record and are not reported.

3.6 Ethics, Privacy, Security and Responsible AI

Only aggregated demand and weather data are used, so no personal or sensitive information is processed, and formal ethical clearance was not considered necessary at proposal stage. The data were obtained from Kaggle, so their use is subject to the

licence and attribution terms of the dataset page. Security of the service is addressed at the API gateway through authentication, validation and rate limiting.

Responsible-AI considerations are practical. The MC Dropout variants report an interval with each forecast so that users do not treat a point value as certain. The intervals are described as empirical intervals that hold under the study's assumptions, not as guarantees for unseen extreme events. The limitations of the evaluation, including undocumented data provenance, are stated openly. Use of generative AI tools in preparing this thesis is disclosed in Appendix B.

3.7 Reproducibility and Research Artefacts

Table 3.3 lists the items needed to reproduce the results and the project artefacts in which they are recorded.

Table 3.3: Reproducibility checklist and status

Item	Status / location
Dataset file, date range, record count	Documented (Section 3.2); `data/electricity_demand_srilanka_fixed.csv`
Lookback, horizon, feature list, split ratios	Documented (Section 3.2, Appendix A)
Train-only scaling	Documented in the notebook protocol
Optimiser, learning rate, batch size, epochs, scheduler, early stopping, seed	Documented (Table 5.2)
Dropout configuration and MC passes	Documented (0.20 final dropout; 100 passes); model-specific rates in notebooks and checkpoints
Model registry and runtime configurations	`api/core/model_registry.py`, `Models/*/runtime/production_config.json`
Calibration constants ($\hat{q}$)	`Models/*_mc/runtime/deployment_config.json`
Final metrics, ranking and statistical comparison	`outputs/final_model_comparison/` (metrics_and_ranking.csv, statistical_comparison.csv)
Interval coverage and width	`Models/*_mc/evaluation/metrics.json`
Integration description	`docs/FINAL_MODEL_INTEGRATION.md`
Hardware, training time and inference-time benchmarks	Not recorded in the project artefacts

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders

Three groups of stakeholders were identified in the project proposal. **Energy planners and analysts** need forecasts and an understanding of demand behaviour for strategic decisions. **Utility companies such as CEB and LECO** need reliable forecasts to avoid blackouts, to manage generation and to reduce waste. **Industrial and commercial consumers** benefit from a stable grid and from information on how their consumption interacts with it. For the forecasting component, the principal users are planners and operators who read forecasts and intervals on the dashboard, and the other project services that consume the forecasts through the API.

4.1.2 Functional requirements

Table 4.1: Functional requirements of the forecasting component

ID	Requirement
FR1	Accept preprocessed time-series data containing weather, temporal, calendar and lag features.
FR2	Produce a demand forecast for the next 15-minute interval from a 96-step window of 20 features.
FR3	Return a calibrated 95% prediction interval together with the forecast for the MC Dropout variants.
FR4	Keep the trained variants in a registry with stable identifiers, rank them under a common protocol and serve the variant that is requested or configured.
FR5	Expose forecasts and intervals through a documented backend API.
FR6	Present forecasts, intervals and model information on the dashboard.

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements of the forecasting component

Attribute	Requirement
Accuracy	Forecast error should be lower than that of the single-family baseline models on the held-out test set.
Reliability	Prediction intervals should achieve close to their nominal coverage so that users can rely on them.
Scalability and latency	Inference, including repeated stochastic passes, should return within a time acceptable for 15-minute operation.
Interpretability	Outputs should be easy to read: a forecast, a range where available, and the variant used.
Flexibility	The component should allow variants to be added, compared and replaced as demand behaviour changes.
Security	Access should be authenticated and inputs validated at the API gateway.

4.2 Component Architecture within the Complete Solution

The place of the forecasting component in the overall architecture was shown in Figure 1.1. At run time the component operates as shown in Figure 4.1. A request arrives at the API gateway and is passed to the feature pipeline, which builds the 96 × 20 input window and applies the training-fitted scaling. The window is passed to the variant named in the request, or to the configured default, drawn from the model registry. A deterministic variant returns a point forecast from a single forward pass. An MC Dropout variant makes 100 stochastic passes, takes their mean as the forecast, and adds and subtracts the stored calibration constant to give a 95% interval. The forecast, the optional bounds and the identity of the variant are returned to the dashboard or consuming service.

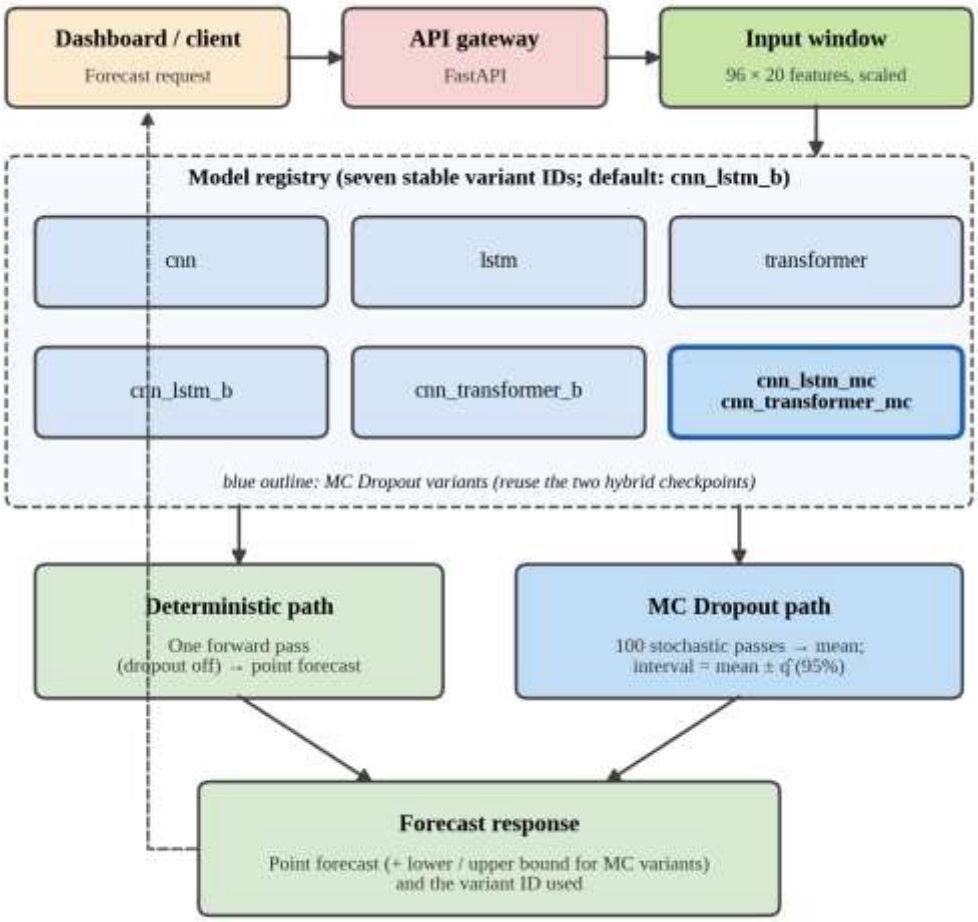


Figure 4.1: Run-time architecture of the adaptive forecasting component

Source: Author's design.

The component interacts with the rest of the system as follows. The preprocessing and feature pipeline and the time-series database supply the data. The model store holds the trained checkpoints and runtime configuration files so that variants can be registered and served without redesigning the service. The application database records users, reports and alerts. The XAI and scenario services consume the forecasts. The behaviour intelligence service could signal changes in consumption behaviour that should prompt re-evaluation of the registry, but no automated link of this kind was found in the project artefacts, so it is treated as future work.

Changes since the proposal. The proposal envisaged an adaptive ensemble with dynamic weighting of a CNN-LSTM and a Transformer, a Bayesian probabilistic

layer, short-term and long-term forecast outputs, and socio-economic input variables. The implemented component provides a registry of seven variants with ranking-based selection, an MC Dropout approximation to Bayesian inference with validation-residual calibration, a one-step 15-minute horizon, and a dataset without socio-economic variables. The narrowing allowed a controlled comparison to be completed on the available data, and the original ambitions are retained as future work (Section 9.4).

4.3 Design Alternatives and Decisions

Table 4.3 records the main design decisions, the alternatives that were available and the reasoning behind each choice.

Table 4.3: Key design decisions and justification

Decision	Alternatives	Choice and rationale
Input window	Shorter or longer windows	96 steps (24 h) covers one complete daily cycle at 15-minute resolution.
Forecast horizon	Multi-horizon outputs	One step (15 min) gives a controlled, clearly evaluated task; multi-horizon is future work.
Validation scheme	Random split; k-fold cross-validation	A chronological split prevents future information reaching training.
Scaling	Scaling on the full series	Training-only fitting avoids test-set leakage.
Model families	Single model only	Compare CNN, LSTM, Transformer and two hybrids to find the best structure empirically.
Uncertainty method	Quantile regression; full Bayesian layers	MC Dropout can be applied to existing checkpoints at inference, without retraining or a change of loss function.
Interval calibration	Raw dropout spread	A validation-residual quantile corrects the under-coverage of the raw spread.
Adaptation	Online learning; automatic model switching	An explicit registry with stable IDs is auditable and avoids silent fallbacks; online learning is left for future work.
Runtime default	Best-ranked variant	The deterministic CNN-LSTM is the configured default (single pass, statistically level with the best model); the choice is a configuration, not an automatic rule.
Serving layer	Notebook-only use	FastAPI delivers the models as a reusable service for the dashboard and other components.

4.4 Detailed Design

4.4.1 Input representation

Each sample is a tensor of shape 96 × 20: 96 consecutive 15-minute time steps, each described by 20 features covering weather, cyclical time encodings, calendar and event indicators, and lagged and rolling demand statistics (Appendix A). The target is the demand in the 15-minute interval that follows the window.

4.4.2 Model families

The **CNN** uses one-dimensional convolutions over the time axis to extract local patterns. The **LSTM** processes the window recurrently and keeps a memory of

longer-term dependence. The **Transformer** applies self-attention to relate every time step to every other. The **CNN-LSTM** passes convolutional features into an LSTM, and the **CNN-Transformer** passes them into attention layers. The baseline architectures share a final dropout layer of 0.20; model-specific dropout rates for other layers are stored in each notebook and checkpoint.

4.4.3 Probabilistic inference

For MC Dropout variants, 100 forward passes are made for each input in the research notebooks. The mean of the passes is the point forecast and their spread characterises raw uncertainty. The cost is about one hundred times that of a single deterministic pass, although the passes can be batched. The two MC variants reuse the checkpoints of the deterministic CNN-LSTM and CNN-Transformer, so no additional training is needed.

4.4.4 Interval calibration

The calibration constant $\hat{q}$ is the empirical 95th percentile of the absolute validation residuals of the MC mean forecast (66.997 kW for the CNN-LSTM and 72.048 kW for the CNN-Transformer). The interval is the MC mean plus and minus $\hat{q}$, so it has constant width. The constants are stored in each variant's deployment configuration and applied at run time.

4.4.5 Registry and selection rule

The registry holds seven stable identifiers: `cnn`, `lstm`, `transformer`, `cnn_lstm_b`, `cnn_transformer_b`, `cnn_lstm_mc` and `cnn_transformer_mc`. Selection follows three rules. First, a request names a variant explicitly, and an unavailable variant is never replaced silently by another. Second, for comparison the variants are ranked by ascending MAE on the common aligned test rows, excluding any variant without saved predictions. Third, when no variant is named, the service uses the configured default, `cnn_lstm_b`. This keeps the behaviour of the service predictable and auditable.

4.4.6 Service interface

The registry is implemented in the FastAPI backend and reads each variant's runtime configuration (`production_config.json` for the deterministic variants and

`deployment_config.json` for the MC variants). A prediction request returns the forecast, the lower and upper bounds where the variant provides them, and the identity of the variant used. The integration is described in the project document `docs/FINAL_MODEL_INTEGRATION.md`.

4.5 Standards and Professional Practice

The design follows established practice for time-series machine learning: chronological validation, leakage prevention, comparison of all models on one protocol, and reporting of multiple error metrics. Software engineering practice follows the team workflow described in the proposal: version control on GitHub with a branch per component and pull-request merges to avoid integration conflicts, a REST/JSON interface between services, and separation of the frontend, API gateway, AI services and data stores. The thesis follows the SLIIT dissertation guidelines for structure and formatting and uses IEEE referencing.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 lists the technologies used for the forecasting component and its integration. The notebooks set the NumPy and TensorFlow random seeds, which confirms TensorFlow as the training framework. PyTorch, mentioned in the proposal as an option for MC Dropout, is not evidenced in the project artefacts. The other libraries are those stated in the proposal.

Table 5.1: Technology stack and justification

Technology	Role and justification
Python 3.10	Main language; the deep-learning ecosystem is built around it
TensorFlow 2.x / Keras	Definition and training of the five architectures; dropout kept active for MC inference
NumPy	Sliding-window sequences and aggregation of the 100 stochastic passes
pandas and scikit-learn	Lag and rolling features, scaling and metric functions (per the proposal)
Google Colab Pro (GPU)	GPU access for training (per the proposal)
FastAPI	Backend API and model registry that serve forecasts to the dashboard and services
React.js and Tailwind CSS	Dashboard with forecast views, metric cards and alerts
GitHub and VS Code	Version control with branch-per-component workflow; development environment

5.2 Component Implementation

Implementation proceeded in the stages below. Each stage produced an artefact (notebook, configuration or model file) that is listed in Appendix A.

1. **Data preparation.** Construction of the 20 features and of 96-step sequences with one-step targets from the 15-minute dataset.
2. **Splitting and scaling.** Chronological 80/10/10 split; scaler fitted on training data and applied to validation and test data.

3. **Model training.** Training of five architectures (CNN, LSTM, Transformer, CNN-LSTM and CNN-Transformer) with a common data pipeline and training protocol.
4. **Uncertainty estimation.** MC Dropout inference with 100 stochastic passes, reusing the CNN-LSTM and CNN-Transformer checkpoints, which gives the two MC variants.
5. **Calibration.** Computation of the 95th-percentile validation residual $\hat{q}$ for each MC variant and storage in its deployment configuration.
6. **Registry and service.** Registration of the seven variants with stable IDs and a configured default, and FastAPI endpoints that load a variant and return forecasts and intervals.

Table 5.2 records the training configuration of the baseline protocol. The two MC research notebooks are checkpoint-reuse and inference comparisons rather than training runs; their recorded training settings (a maximum of 100 epochs for the CNN-LSTM and 50 for the CNN-Transformer) are protocol metadata.

Table 5.2: Training configuration of the baseline protocol

Setting	Value
Optimiser and initial learning rate	Adam, 0.001
Loss function	Mean absolute error (MAE)
Batch size	128; time-series batches not shuffled
Maximum epochs	50 (actual epochs can be fewer because of early stopping)
Early stopping	Patience 15 epochs
Learning-rate schedule	ReduceLROnPlateau: patience 8, factor 0.5, minimum 1e-5
Dropout	Final dropout layer 0.20 in the baseline architectures; 0.20 recorded for the MC variants
MC Dropout passes	100 in the research notebooks; the runtime configuration can differ
Random seed	42 (NumPy and TensorFlow)
Hardware and training time	Not recorded in the project artefacts

5.3 Integration with the Other Components

The forecasting component is integrated through a FastAPI backend that sits behind the API gateway and hosts the model registry. The preprocessing and feature pipeline provides the input data, the dashboard requests and displays forecasts and intervals, and the XAI and scenario services call the forecast service when they need predictions. The end-to-end path is: data input, preprocessing, sequence generation, registry lookup, inference (deterministic or MC), calibration, API response and dashboard or consuming service. Turning the models from isolated notebooks into a service allows the other components to use them without reproducing the preprocessing or model code.

5.4 Verification and Software Testing

Table 5.3 lists the verification checks that apply to the data pipeline, the models and the service, and the evidence available for each. No automated test report for the API or dashboard was found in the project artefacts, so service-level testing is not claimed.

Table 5.3: Verification checks and available evidence

Area	Check	Evidence
Data pipeline	Train, validation and test segments are strictly chronological and non-overlapping	Documented in the notebook protocol
Data pipeline	Scaler is fitted on training data only	Documented in the notebook protocol
Data pipeline	Window tensors are 96 × 20 and the target is the next step	Documented in the protocol
Models	Variants are compared on identical aligned test rows	Six variants on 18,817 aligned rows
Models	Repeated runs with different seeds are consistent	Not done: single seed (42)
Uncertainty	MC Dropout is active at inference and calibrated coverage is checked	Coverage on validation and test (Section 6.1)
Integration	Dashboard displays the forecast and interval end to end	Recorder

5.5 Deployment and Operation

The backend and explanation services are hosted on a cloud server, with a domain and SSL certificate providing secure access to the dashboard and API, and cloud storage holding the demand data, model outputs and related research data (see the budget in Table 8.1). Operationally, the main considerations are the cost of 100 stochastic passes per MC request, the need to monitor forecast error against actual demand after deployment, and the need to re-evaluate and replace variants as conditions change. Monitoring and scheduled re-evaluation are recommended, but no automated mechanism for them exists in the project artefacts.

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Experimental setting

All results are for 15-minute Sri Lankan demand, a 96-step window of 20 features, a one-step horizon and a chronological 80/10/10 split with training-only scaling. Six variants have saved predictions that can be aligned on the same **18,817-row** test set, and their metrics were recalculated from those aligned rows. The CNN-LSTM with MC Dropout has no saved prediction file and therefore cannot be compared on the aligned set; its calibrated interval results are reported separately in Section 6.1.5. An earlier project record gave an MAE of 21.16 kW for this variant, but that figure cannot be reproduced from the available files and is not used.

6.1.2 Model comparison

Table 6.1 gives the point-forecast metrics of the six comparable variants, ranked by MAE, and Figure 6.1 illustrates the comparison.

Table 6.1: Test performance of the evaluated variants on the aligned test set (18,817 rows)

Rank	Model variant	MAE (kW)	RMSE (kW)	MAPE (%)	R^2
1	**CNN-Transformer MC**	**20.46**	**29.23**	**0.97**	**0.9745**
2	CNN-LSTM-B	20.52	29.98	0.98	0.9732
3	LSTM	30.72	42.88	1.47	0.9451
4	Transformer	32.61	42.00	1.54	0.9473
5	CNN-Transformer-B	38.80	46.38	1.76	0.9357
6	CNN	39.46	52.37	1.78	0.9181
7	CNN-LSTM MC	21.16	30.26	1.01	0.9726

Source: Project experiments; metrics recalculated from aligned prediction rows. CNN-LSTM MC is not comparable because its prediction file is unavailable.

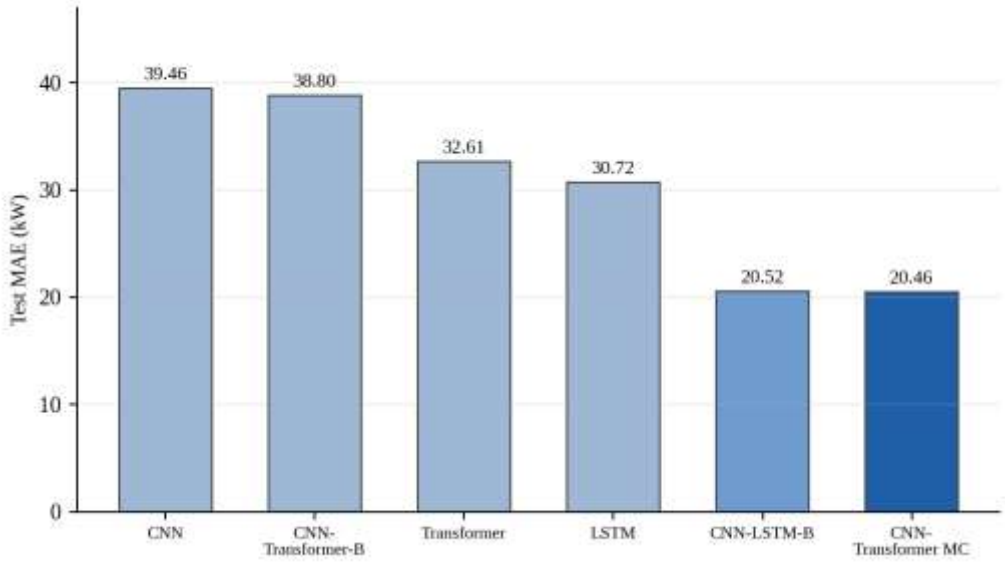


Figure 6.1: Test MAE by model variant on the aligned test set (lower is better)

Source: Project experiments.

Two variants form a clear leading group: the CNN-Transformer with MC Dropout (MAE 20.46 kW) and the deterministic CNN-LSTM (20.52 kW). They are followed by the LSTM (30.72 kW), the Transformer (32.61 kW), the deterministic CNN-Transformer (38.80 kW) and the CNN (39.46 kW). Note that the deterministic CNN-Transformer, which uses the same checkpoint as the leading MC variant, is nearly as poor as the CNN alone. This is discussed in Section 7.1.

6.1.3 Relative improvements and significance

Table 6.2 expresses the best MAE as a relative reduction against each other comparable variant. The percentages are derived from Table 6.1.

Table 6.2: MAE reduction achieved by the CNN-Transformer with MC Dropout (20.46 kW)

Compared with	MAE (kW)	Difference (kW)	Relative reduction
CNN-LSTM-B	20.52	0.06	0.3%
LSTM	30.72	10.26	33.4%
Transformer	32.61	12.15	37.3%
CNN-Transformer-B	38.80	18.34	47.3%
CNN	39.46	19.00	48.2%

Source: Derived by the author from Table 6.1.

A paired moving-block comparison of the CNN-Transformer with MC Dropout against the deterministic CNN-LSTM gives an MAE difference of −0.0613 kW ($p = 0.9041$). This does not establish a meaningful performance difference: the two leading models are statistically indistinguishable. The improvement over the single-family models is, by contrast, large (33.4% against the LSTM and 37.3% against the Transformer). Hypothesis H2 (the CNN-Transformer outperforms the CNN-LSTM) is therefore not supported, and hypothesis H1 is only partly supported, because one hybrid, the deterministic CNN-Transformer, does not beat the single-family LSTM and Transformer.

The effect of MC inference on one checkpoint is striking. The deterministic CNN-Transformer (38.80 kW) and the MC variant (20.46 kW) use the same trained weights, and averaging 100 stochastic passes cuts the MAE by 47.3%.

The ratio of RMSE to MAE for the best model is 29.23 / 20.46 = 1.43, higher than the value of about 1.25 expected for normally distributed errors, which indicates that some larger errors are present. R^2 of 0.9745 means that the model explains about 97.5% of the variance in the test series.

6.1.4 Peak-period performance

Table 6.3 and Figure 6.2 give the MAE on observations at or above the 90th percentile of the aligned test load. The deterministic CNN-LSTM has the lowest peak error (6.83 kW), ahead of the CNN-Transformer with MC Dropout (9.49 kW) and the LSTM (9.73 kW), while the CNN and the deterministic CNN-Transformer have very large peak errors (56.47 kW and 46.87 kW). Peak error is lower than overall MAE for every variant except the CNN and the deterministic CNN-Transformer, so for those four variants errors are not concentrated in high-demand periods. No separate holiday-only metric was computed, and the peak metric must not be read as a holiday result.

Table 6.3: Peak (top-decile) MAE on the aligned test set

Model variant	Peak MAE (kW)	Overall MAE (kW)
CNN-LSTM-B	6.83	20.52
CNN-Transformer MC	9.49	20.46
LSTM	9.73	30.72
Transformer	15.68	32.61
CNN-Transformer-B	46.87	38.80
CNN	56.47	39.46
CNN-LSTM MC	Not available (no saved predictions)	

Source: Project experiments (outputs/final_model_comparison/metrics_and_ranking.csv).

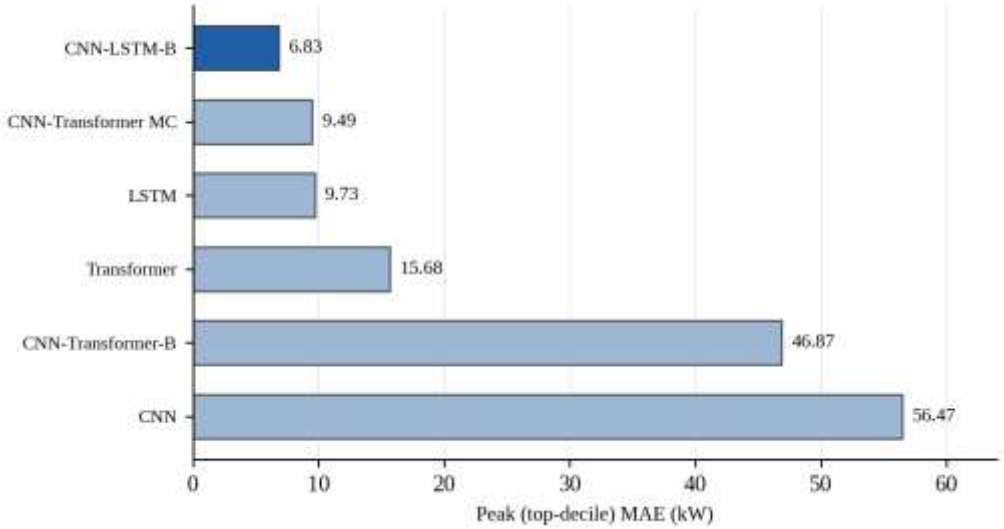


Figure 6.2: Peak (top-decile) MAE by model variant

Source: Project experiments.

6.1.5 Uncertainty and interval quality

Only the two MC variants have interval results. The nominal level is 95% ($\alpha = 0.05$) and the calibration constants are $\hat{q} = 66.997$ kW (CNN-LSTM) and 72.048 kW (CNN-Transformer). Table 6.4 compares the raw MC interval (mean ± 1.96 standard deviations) with the calibrated interval, and Figure 6.3 shows the coverage.

Table 6.4: Raw and calibrated 95% interval coverage and mean width

Model	Interval	Validation coverage (%)	Validation width (kW)	Test coverage (%)	Test width (kW)
CNN-LSTM MC	Raw MC ± 1.96 SD	74.39	103.00	85.23	142.22
CNN-LSTM MC	Calibrated ± $\hat{q}$	95.01	133.99	96.33	133.99
CNN-Transformer MC	Raw MC ± 1.96 SD	85.88	116.88	90.83	157.49
CNN-Transformer MC	Calibrated ± $\hat{q}$	95.01	144.10	97.12	144.10

Source: Project experiments (evaluation metrics.json files).

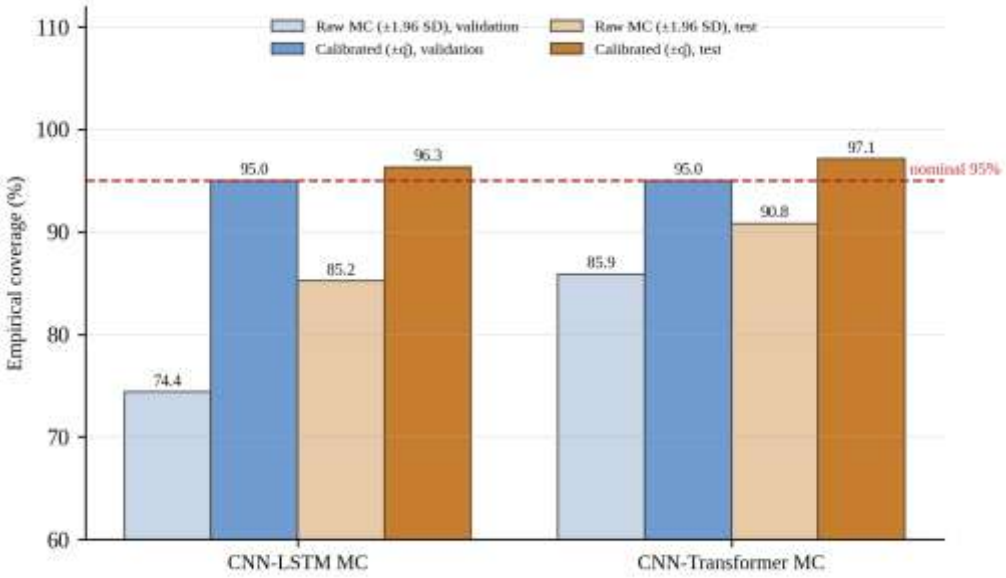


Figure 6.3: Empirical coverage of raw and calibrated 95% intervals

Source: Project experiments.

Three conclusions follow. First, the raw MC spread under-covers badly (74-86% on validation and 85-91% on test against a 95% target), which confirms that dropout spread alone is not a reliable interval. Second, the calibrated interval reaches 95.01% on validation by construction and 96.33% and 97.12% on the test set, so it is slightly conservative on unseen data. Third, the calibrated width is constant (134 kW for the CNN-LSTM and 144 kW for the CNN-Transformer), about 3.3 and 3.5 times the best MAE on either side of the forecast, so it does not widen in volatile periods or tighten in calm ones. Because validation data were also used for early stopping, these

coverage figures are supplementary and are not claimed as strict conformal guarantees. No interval results exist for the five deterministic variants.

6.1.6 Predictions on the test set

A time-series plot of actual and predicted demand would let readers see the behaviour of the models directly. The prediction files were not available to the thesis at the time of writing, so Figure 6.4 is a placeholder to be completed from the saved prediction rows.

6.2 Product and Technical Results

The forecasting capability has been delivered as a service. The FastAPI backend hosts a registry of seven variants with stable identifiers, returns forecasts (and 95% intervals for the MC variants) and uses the deterministic CNN-LSTM as its configured default. This default is a sensible operating point: it needs a single forward pass, its error is statistically level with the best model, and it has the lowest peak error, while the MC variants are available when an interval is required. Comparable inference-time benchmarks were not supplied. The MC variants need about 100 forward passes per request, so their latency is substantially higher than that of a deterministic variant.

6.3 User or Stakeholder Validation

No stakeholder validation protocol, participant records, acceptance criteria or sign-off evidence was found in the project artefacts, so no stakeholder validation is claimed. The usefulness of the dashboard and the intervals for planners and operators has therefore not been tested. A usability evaluation with planners or operators is recommended as future work.

6.4 Integration Results

The forecast service has been connected to the backend and dashboard, so that forecasts and intervals can be requested and displayed through the normal path of the system, as described in `docs/FINAL_MODEL_INTEGRATION.md`. The registry resolves variants by stable identifier, reads their runtime configuration, and does not fall back

silently to another model if one is unavailable. Formal integration-test records with the other three components were not found.

6.5 Summary of Findings

Table 6.5: Summary of findings and strength of evidence

No.	Finding	Evidence strength
F1	The CNN-Transformer with MC Dropout (MAE 20.46 kW, RMSE 29.23 kW, MAPE 0.97%, R^2 0.9745) and the deterministic CNN-LSTM (20.52 kW) are the two best variants and are statistically indistinguishable (difference −0.06 kW, p = 0.90).	Strong for the tie; ranking between them is not established
F2	Both leading variants beat the single-family models by about one-third or more (LSTM 30.72, Transformer 32.61, CNN 39.46 kW).	Strong: large margin on a common test set
F3	The deterministic CNN-Transformer (38.80 kW) is no better than the single-family models; MC inference on the same checkpoint cuts its MAE by 47.3%.	Strong as an observation; the cause is not isolated
F4	The deterministic CNN-LSTM has the lowest peak-period MAE (6.83 kW).	Moderate: one metric, one test set
F5	Raw MC dropout intervals under-cover; calibration gives 95.0% on validation and 96.3-97.1% on test, with constant width (134-144 kW).	Moderate: validation also used for early stopping
F6	Forecasts and intervals are served through a FastAPI model registry to the dashboard.	Demonstrated; no formal test records
F7	No stakeholder validation, holiday-only metric or inference-time benchmark exists.	Not available

7 DISCUSSION

7.1 Interpretation of Findings

The central result is that two variants, the CNN-Transformer with MC Dropout (MAE 20.46 kW) and the deterministic CNN-LSTM (20.52 kW), are far more accurate than the single-family LSTM, Transformer and CNN (30.72 to 39.46 kW), and that the two leaders cannot be separated statistically (difference −0.06 kW, $p = 0.90$). The finding supports combining convolutional feature extraction with a sequence model, but it does not show that one hybrid is better than the other. The following interpretations are plausible; none was isolated by a dedicated ablation study, so they are hypotheses.

- **Local and long-range structure both matter.** The CNN alone, which sees only local patterns, is the weakest model (39.46 kW), whereas the LSTM, which models the whole window, is clearly better (30.72 kW). Adding convolutional feature extraction in front of the LSTM roughly cuts the error by a further third (20.52 kW).
- **The standalone Transformer is competitive with the LSTM but not better.** At 32.61 kW it sits just behind the LSTM, consistent with the view that self-attention over a short, noisy window offers no advantage without a local front end.
- **The deterministic CNN-Transformer is an anomaly.** At 38.80 kW it is no better than the CNN and much worse than the LSTM, yet the same checkpoint reaches 20.46 kW when 100 dropout-perturbed passes are averaged. Averaging many stochastic sub-networks behaves like an ensemble and may smooth the variance of a model whose single deterministic pass is poorly behaved, for example because of a mismatch between training-time dropout and inference-time behaviour. A capacity-matching CNN-Transformer variant (V2) also underfitted and was rejected, which suggests that this architecture is sensitive to configuration. The advantage of the CNN-Transformer therefore depends heavily on MC inference rather than on the architecture alone.

- **Peak error and overall error need not agree.** The deterministic CNN-LSTM has the lowest peak MAE (6.83 kW) although its overall MAE is marginally higher than that of the best model, so it is the stronger choice if high-demand periods are the priority.

On interval quality, the raw MC spread under-covered (74-86% on validation) and had to be rescaled with a validation-residual quantile. After calibration the intervals reached 95.0% on validation and 96.3-97.1% on test. The slight over-coverage on test is conservative and therefore safe, but the constant width (134-144 kW, roughly 3.3-3.5 times the MAE) means the interval carries no information about whether a given period is more or less uncertain. The approach delivers reliable but coarse uncertainty.

Two practical consequences follow. The configured default (the deterministic CNN-LSTM) is justified: it is statistically level with the best model, needs one forward pass and has the best peak error. The MC variants are worth their roughly 100-fold inference cost only when an interval is needed.

7.2 Comparison with Literature Baselines and Existing Solutions

Results from other studies cannot be compared numerically with those reported here because the datasets, horizons, spatial levels and metrics differ. The findings are, however, consistent in direction with the literature: deep-learning and hybrid models improve on traditional or single models in Sri Lankan and international studies [5], [6], [4], [15], and the benefit of combining methods reported for Prophet-XGBoost [4] is mirrored here in a deep-learning setting. The finding that the LSTM was the strongest single-family model agrees with the report of [6] that LSTM gave the lowest errors in its comparison.

Relative to existing Sri Lankan solutions, which mainly provide point forecasts from LSTM or MLP models, this component adds a comparison of hybrid architectures, calibrated prediction intervals and a deployable service. Its input features include weather variables, which studies such as [14] show to be influential, but it omits the socio-economic variables that the proposal had planned to use.

7.3 Research Contribution

The research contribution is a consistent chronological evaluation of seven model variants on 15-minute demand data, evidence that hybrid CNN-based models substantially outperform single-family models on this task while the two leading hybrids are statistically tied, the observation that MC inference transforms the accuracy of one checkpoint (47% MAE reduction), and a calibrated interval pipeline that corrects the under-coverage of raw MC Dropout. The thesis also states precisely what the adaptive component is and is not, which supports honest comparison with future work.

7.4 Product Contribution

The product contribution is a forecasting service with a registry of seven stable model variants. It returns a 15-minute-ahead forecast and, for the MC variants, a calibrated 95% interval, through FastAPI and the dashboard. It is a reusable building block for the XAI and scenario services and for planners who need a range rather than a single number.

7.5 Limitations and Threats to Validity

Table 7.1: Limitations and threats to validity

Type	Limitation	Consequence and mitigation
External	The dataset is a Kaggle-hosted CSV whose original publisher, collection method and spatial level are not documented	Its relationship to real CEB demand cannot be verified; confirm provenance with the dataset page and, if possible, validate on utility data.
External	No socio-economic variables (GDP, electricity price) although the proposal planned them; weather values are historical observations	Limited ability to model structural change; add economic variables and use weather forecasts operationally.
Internal	Single chronological split and a single random seed (42)	Run-to-run variability from re-training was not measured; use walk-forward evaluation and repeated seeds.
Internal	Validation data were used for both early stopping and interval calibration	Calibrated coverage is supplementary, not a strict conformal guarantee; use a separate calibration set.
Conclusion	Only the two leading models were compared with a paired test; the CNN-LSTM with MC Dropout has no saved predictions	Other pairwise differences and one variant are not tested; keep and compare all prediction files.
Construct	No holiday-only metric; the peak metric is a top-decile MAE; MAPE is unstable near zero	Event-specific performance is unknown; add holiday and Poya-day subsets.
Method	Lag and rolling features replace a raw demand channel; their alignment must be verified	A misalignment could leak information; confirm in the feature code.
Method	One-step (15-minute) horizon only	Does not show performance at longer lead times; add multi-horizon forecasting.
Method	MC Dropout is an approximation; the interval has constant width	Intervals may be unreliable under distribution shift and are not heteroscedastic; monitor coverage and consider adaptive intervals.
Method	About 100 passes per MC forecast; no inference-time benchmark	Latency not quantified; benchmark and consider fewer passes.
Method	Adaptation is a registry with explicit default, not online learning or automatic selection	The system does not learn from new data on its own; add drift detection and retraining.
Validation	No stakeholder validation was documented	Usefulness for planners is untested; carry out a usability evaluation.

7.6 Implications

For planners and utilities, the results suggest that a convolutional-recurrent forecaster can give materially better 15-minute forecasts than single-family models, and that calibrated ranges can inform reserve margins. For researchers, the study illustrates the value of a controlled comparison, of statistical testing between the leading models, and of calibrating neural-network uncertainty. For practitioners, it shows that interval coverage, peak-period error, inference cost and the provenance of the data should be examined alongside overall accuracy.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The proposition is an adaptive, uncertainty-aware forecasting service tailored to Sri Lanka that moves planners from a single point estimate to a calibrated range. The primary customer persona is the *grid strategist*: an operations engineer or lead planner at a utility such as CEB, who needs better prediction of load, cannot currently judge forecast risk, and wants to reduce the cost of reserves and the chance of load shedding. The forecasting component supplies the numerical core, and the wider system adds behavioural insight, explanation and scenario analysis.

8.2 Market and Competitive Position

The primary market is national utilities and grid operators in developing countries, beginning with Sri Lanka and extending to South Asia. Secondary markets are large industrial zones, renewable-energy developers and smart-city infrastructure planners. The competitive advantages claimed in the proposal are regional specialisation (local holidays, Poya days, tropical weather and economic cycles) and the provision of probabilistic ranges rather than single numbers. No formal market study or customer interviews have been carried out, so these claims are hypotheses for a pilot to test.

8.3 Business or Sustainability Model

The proposed revenue model is business-to-government (B2G) licensing as an annual software-as-a-service subscription to grid operators, supplemented by one-off consultancy and integration fees for connecting the module to existing SCADA or smart-meter infrastructure. Pricing is intended to be value-based, linked to savings the utility can demonstrate, with discounted pilot programmes for provincial utilities to collect proof of concept and refine local models. Quantifying the savings from a given improvement in forecast accuracy requires utility cost data that the project does not have, so no savings figure is asserted here.

Running costs comprise cloud computing for training and serving deep-learning models, data storage and hosting, and the effort to monitor forecasts and retrain

models as consumption behaviour drifts. Table 8.1 gives the project budget planned in the proposal, which indicates the order of magnitude of development cost.

Table 8.1: Planned project budget (LKR)

Item	Quantity / duration	Unit cost	Total
Hardware	1	217,000	217,000
Cloud computing (Google Colab Pro / GPU)	9 months	4,500	40,500
Web hosting / cloud server	9 months	2,500	22,500
Data storage (cloud)	9 months	1,500	13,500
Domain and SSL certificate	1	5,000	5,000
Visualisation and analytics tools	1	6,000	6,000
Data collection travel	4 trips	800	3,200
Miscellaneous	1	4,000	4,000
Total			**311,700**

Source: Project proposal, R26-IT010.

8.4 Scalability, Intellectual Property and Compliance

Scalability. The service can scale by serving several utilities or regions from separate model artefacts held in the model store, and by running the repeated MC Dropout passes in batches on GPUs. Inference cost could be reduced by lowering the number of passes or using cheaper uncertainty approximations.

Intellectual property. The proposal considered protecting the hybrid model structures and the explainable-AI reporting and scenario interfaces. Because CNN, LSTM and Transformer layers are well-known building blocks, the strongest protection is likely to lie in the source code, trained models, data pipeline and local calibration rather than in the architectures themselves. Ownership under SLIIT policy and any data-provider agreements should be confirmed.

Compliance. The system uses aggregated, anonymised data. Any commercial deployment should respect applicable data-protection law (for example Sri Lanka's Personal Data Protection Act, No. 9 of 2022), contractual limits on the use of utility

data, and the requirements of the sector regulator. Forecasts should be presented with their intervals and limitations so that they are not over-relied upon in safety-related decisions.

8.5 Contribution to the Sustainable Development Goals

- **SDG 7 (Affordable and Clean Energy).** Better short-term forecasts support reliable and efficient supply and make it easier to plan reserves and integrate variable renewable generation.
- **SDG 9 (Industry, Innovation and Infrastructure).** The component applies advanced AI methods (hybrid deep learning, uncertainty quantification) to critical energy infrastructure.
- **SDG 13 (Climate Action).** More accurate demand forecasts reduce over-generation from fossil-fuel plants and so can lower unnecessary emissions. The size of this effect has not been measured in this study.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Adaptive Forecasting and Prediction component of the AI-Based Electricity Demand Intelligence System for Sri Lanka. Using a 15-minute dataset of 189,888 records (January 2020 to May 2025), 20 weather, temporal, calendar and lag features, a 96-step window and a one-step horizon, with a chronological split and training-only scaling, five architectures were trained and seven variants evaluated. On an aligned test set of 18,817 rows, the CNN-Transformer with MC Dropout achieved the lowest error (MAE 20.46 kW, RMSE 29.23 kW, MAPE 0.97%, R^2 0.9745), but it is statistically indistinguishable from the deterministic CNN-LSTM (20.52 kW, $p = 0.90$), which has the lowest peak-period error. Both are far more accurate than the single-family LSTM, Transformer and CNN. MC inference cut the error of the CNN-Transformer checkpoint by 47%.

Raw MC Dropout intervals under-covered, and a validation-residual calibration constant brought 95% intervals to 95.0% coverage on validation and 96.3-97.1% on test, at constant widths of 134-144 kW. The variants are served through a FastAPI registry to the dashboard, with the deterministic CNN-LSTM as the configured default. The adaptive element is a switchable, ranked model registry rather than automatic selection or online learning. Stakeholder validation, a holiday-specific evaluation, multi-seed repetition and documented data provenance are outstanding. Within those bounds, the work provides a reproducible comparison protocol, a calibrated probabilistic forecaster and a usable service for Sri Lankan short-term demand forecasting.

9.2 Achievement of Objectives

Table 9.1: Achievement of the objectives

Objective	Status	Evidence
1. Leakage-safe data pipeline	Achieved	Chronological 80/10/10 split; training-only scaling (Section 3.2); feature alignment to be confirmed
2. Train five architectures and evaluate seven variants	Achieved	Table 6.1; six variants compared on 18,817 aligned rows, the CNN-LSTM MC variant evaluated separately
3. MC Dropout and calibrated 95% intervals	Achieved with caveats	100 passes; 95.0% validation and 96.3-97.1% test coverage; validation also used for early stopping
4. Model registry, ranking and default	Achieved in limited form	Seven stable IDs, MAE ranking, default cnn_lstm_b; no automatic selection or online learning
5. FastAPI and dashboard integration	Achieved	Service and dashboard integration (Chapter 5); formal test records not found
6. Critical evaluation and limitations	Achieved	Sections 6.1 and 7.5

9.3 Individual Research and Product Contributions

The research contribution is the development and evaluation of a short-term demand-forecasting framework that compares five architectures and seven variants under a consistent chronological protocol, tests the two leading models statistically, integrates uncertainty estimation through MC Dropout, and calibrates prediction intervals from validation residuals. The product contribution is the integration of this framework through a FastAPI model registry and dashboard so that it operates as a practical forecasting service within the wider intelligence system.

9.4 Future Directions

- Continuous or periodic retraining with newly observed demand, and true online or adaptive learning with drift detection.
- Multi-horizon forecasting (for example 30 minutes, 1 hour, 6 hours and 24 hours) and an adaptive ensemble with dynamic weighting as originally proposed.

- Integration of socio-economic variables (for example GDP and electricity price) and of weather forecasts, and validation on utility-supplied CEB data.
- Repeated runs with several random seeds, walk-forward evaluation and a separate calibration set to give strict conformal coverage.
- Holiday-, Poya-day- and event-specific evaluation, and adaptive-width intervals that widen in volatile periods.
- An ablation study of why MC inference transforms the CNN-Transformer, and inference-time benchmarking and model compression to reduce the cost of MC passes.
- A usability evaluation with planners and operators, and automated monitoring of forecast error and interval coverage after deployment.

REFERENCES

[1] International Energy Agency (IEA), "Electricity 2024: Analysis and forecast to 2026," IEA, Paris, France, 2024.

[2] Ceylon Electricity Board, "Annual Report 2022," Ceylon Electricity Board, Colombo, Sri Lanka, 2022.

[3] Ceylon Electricity Board, "CEB Care portal." [Online]. Available: https://cebcare.ceb.lk/gensum/details. [Accessed: Mar. 10, 2026].

[4] R. K. Shiwakoti et al., "Enhancing short term electricity demand forecasting: A hybrid Prophet-XGBoost approach for improved accuracy," 2026.

[5] A. Jayasinghe et al., "Electricity load/demand forecasting in Sri Lanka using deep learning techniques," in *Proc. 2021 10th Int. Conf. Information and Automation for Sustainability (ICIAFS)*, 2021.

[6] R. K. Shiwakoti et al., "Short-term electricity demand forecasting in Sri Lanka using statistical and deep learning models," in *Proc. World Construction Symposium*, 2025.

[7] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *International Journal of Forecasting*, vol. 32, no. 3, pp. 914-938, 2016.

[8] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, art. 44, 2014.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.

[10] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[11] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1050-1059.

[12] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," *Journal of the American Statistical Association*, vol. 113, no. 523, pp. 1094-1111, 2018.

[13] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[14] A. D. A. D. Priyadarshana et al., "Using weather patterns to forecast electricity consumption in Sri Lanka: An ARDL approach," *International Energy Journal*, vol. 26, no. 1, pp. 257-268, 2021.

[15] M. G. I. U. Karunarathne et al., "Improving microgrid energy demand forecasting using convolutional neural networks," *Engineer: Journal of the Institution of Engineers, Sri Lanka*, vol. 57, no. 4, pp. 37-45, 2024.

[16] A. Nazir et al., "Forecasting energy consumption demand of customers in smart grid using Temporal Fusion Transformer (TFT)," *Results in Engineering*, vol. 17, 2023.

[17] J. Jenson et al., "Electricity demand forecasting using LSTMs," *Journal of Electrical Engineering and Automation*, vol. 5, no. 2, pp. 228-237, 2023.

[18] D. Raj et al., "Electricity demand forecasting using ML," in *Proc. IEEE*, 2023.

[19] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75-105, 2004.

[20] "Sri Lanka electricity demand dataset (electricity_demand_srilanka.csv)," Kaggle. [Online]. Available: [https://www.kaggle.com/datasets/isuranga/load-forecasting-dataset/data]. [Accessed: [2025]].

GLOSSARY

Chronological split – Division of a time series into training, validation and test segments in time order, so that models are never trained on data later than the data they are tested on.

Concept drift – Change over time in the relationship between inputs and the target, which degrades the accuracy of a fixed model.

Conformal prediction – A method that turns the errors of a model on held-out data into prediction intervals with a stated coverage level.

Data leakage – Use of information from outside the training period (for example test-set statistics) when building a model, which makes results look better than they are.

Lookback window – The number of past time steps given to the model as input; here 96 steps of 15 minutes (24 hours).

Model registry – A table of trained model variants with stable identifiers from which the forecasting service loads the variant that is requested or configured.

Monte Carlo Dropout – Keeping dropout active at prediction time and repeating the prediction many times to obtain a spread that approximates model uncertainty.

Peak error – The MAE computed only on observations at or above the 90th percentile of the actual test load.

Poya day – A monthly Buddhist full-moon public holiday in Sri Lanka, which affects electricity demand patterns.

Prediction interval – A range expected to contain the actual value with a stated probability.

Self-attention – A mechanism by which each position in a sequence weighs its relevance to all other positions.

Walk-forward validation – Repeatedly training on the past and testing on the next block of time, moving forward through the series.

APPENDICES

Appendix A Extended Results and Evidence Record

This appendix lists the dataset requirements stated in the project proposal, the 20 features actually used, and the project artefacts that hold the evidence reported in this thesis.

A.1 Dataset requirements stated in the project proposal

Table A.1: Dataset requirements stated in the project proposal

Group	Variables requested
Temporal	Timestamp (15-minute); hour of day; day of week; month; season
Target	Load demand (kW)
Weather and environment	Temperature (°C); humidity (%); wind speed (m/s); rainfall (mm); solar irradiance (W/m²)
Socio-economic and pricing	GDP (USD); per-capita energy use (kWh); electricity price (LKR/kWh)
Calendar and events	Public event flag (1 = yes, 0 = no)

Source: Project proposal, Appendix A. The dataset used contains the weather, calendar and target variables but no socio-economic variables; the feature set actually used is listed in Table A.2.

A.2 The 20 input features

Table A.2: The 20 input features in the order required by the models

No.	Feature	Description	Group
1	Temperature (°C)	Air temperature	Weather
2	Humidity (%)	Relative humidity	Weather
3	Wind Speed (m/s)	Wind speed	Weather
4	Rainfall (mm)	Rainfall	Weather
5	Solar Irradiance (W/m²)	Solar irradiance	Weather
6	sin_hour	Sine component of the cyclical hour-of-day encoding	Temporal
7	cos_hour	Cosine component of the hour-of-day encoding	Temporal
8	sin_dow	Sine component of the day-of-week encoding	Temporal

No.	Feature	Description	Group
9	cos_dow	Cosine component of the day-of-week encoding	Temporal
10	sin_month	Sine component of the month-of-year encoding	Temporal
11	cos_month	Cosine component of the month-of-year encoding	Temporal
12	is_weekend	Binary weekend indicator	Temporal
13	year_norm	Normalised year index (long-term trend)	Temporal
14	Public Event	Binary public-event indicator	Calendar / event
15	Poya Day	Binary Poya-day indicator	Calendar / event
16	load_lag_96	Demand lagged by 96 steps (24 hours)	Lag / rolling
17	load_lag_192	Demand lagged by 192 steps (48 hours)	Lag / rolling
18	load_lag_672	Demand lagged by 672 steps (7 days)	Lag / rolling
19	load_mean_week	Rolling mean of demand over a week	Lag / rolling
20	load_std_day	Rolling standard deviation of demand over a day	Lag / rolling

A.3 Evidence sources in the project repository

Table A.3: Project artefacts that hold the reported evidence

Evidence	Artefact
Dataset	`data/electricity_demand_srilanka_fixed.csv`
Model registry and stable IDs	`api/core/model_registry.py`
Runtime configurations (deterministic variants)	`Models/*/runtime/production_config.json`
Deployment configurations and $\hat{q}$ (MC variants)	`Models/cnn_lstm_mc/runtime/deployment_config.json`, `Models/cnn_transformer_mc/runtime/deployment_config.json`
Interval coverage and width	`Models/cnn_lstm_mc/evaluation/metrics.json`, `Models/cnn_transformer_mc/evaluation/metrics.json`
Aligned metrics, ranking and peak MAE	`outputs/final_model_comparison/metrics_and_ranking.csv`
Paired moving-block comparison	`outputs/final_model_comparison/statistical_comparison.csv`
Integration description	`docs/FINAL_MODEL_INTEGRATION.md`